MODULE 3 · LECTURE 13

CS 6530 Applied Cryptography

Secure Use of RSA

Module 3 – Public-Key Cryptography

Why RSA Needs Secure Encoding

RSA gives us a powerful mathematical primitive. How do we turn it into a secure encryption scheme?

1977/78 — RSA

Rivest · Shamir · Adleman

Trapdoor public-key construction

$m \to m^e \bmod n$

→

1982/84

Goldwasser–Micali

Probabilistic encryption — stronger notions of encryption security

→

1994

Bellare–Rogaway

OAEP — randomized encoding for trapdoor permutations

→

Secure RSA

Encryption

$M \to (OAEP) \to EM \to (RSA) \to C$

RSA primitive + appropriate randomized encoding → secure encryption scheme

Encryption	Digital Signatures
RSA + OAEP	RSA + PSS

This is only a preview; PSS is explained later.

INSIGHT

RSA was not simply "wrong and later fixed." Cryptographic research progressively strengthened what we require from a secure encryption scheme.

Key idea: A mathematical trapdoor primitive is a building block; secure deployment requires a cryptographic scheme constructed around it.

RSA-OAEP — Randomize Before RSA

TEXTBOOK RSA

$M \rightarrow (RSA) \rightarrow C$

$M \rightarrow (RSA) \rightarrow C$

Same M → same C

RSA itself supplies no fresh randomness.

Rivest-Shamir-Adleman Optimal Asymmetric Encryption Padding (RSA-OAEP)

Inputs: M, fresh random seed, optional L

↓

OAEP ENCODE

↓

$EM \rightarrow (RSA) \rightarrow C$

$(M, L, seed_1) \rightarrow EM_1 \rightarrow C_1$

$(M, L, seed_2) \rightarrow EM_2 \rightarrow C_2$

$C_1 \neq C_2$

for fresh, different seeds — overwhelmingly.

What Does "Encoding" Mean?

OAEP transforms the application message into a randomized, structured representation (EM) suitable as input to the RSA primitive.

M → EM

OPTIONAL LABEL (L)

lHash = Hash(L)

Non-secret application/protocol context. Sender and receiver use the same L (often L="").

HOW MUCH CAN RSA-OAEP CARRY?

$mLen \leq k - 2 \cdot hLen - 2$

RSA-2048 + SHA-256:

$256 - 2(32) - 2 = 190$ bytes

This limitation will motivate hybrid encryption.

Key idea: RSA-OAEP does not change RSA mathematics — it changes what we safely give to RSA.

From Message M to RSA Input — Building EM

1 · BUILD THE DATA BLOCK - DB

`lHash = Hash(L)`

DB

lHash	PS	01	M
32 B	zero octets	1B	message

`DB = lHash ‖ PS ‖ 01 ‖ M`

lHash — binds expected label/context
PS — fills available OAEP space
01 — delimiter; marks where M begins

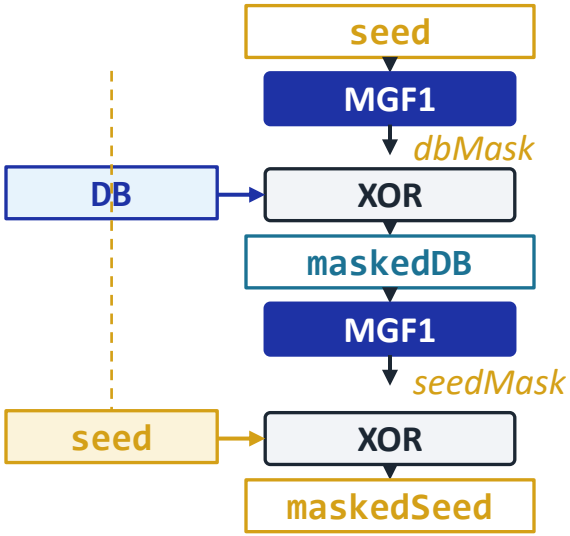
PS is not used to make DB a multiple of the hash length.

MGF1 — Mask Generation Function

$H(X\|C_0)\|H(X\|C_1)\|\cdots$

Extended until mask length is met — similar to HKDF-Expand, but not a KDF.

2·3 · THE TWO MASKS



`EM = 00‖maskedSeed‖maskedDB`

Numerical Grounding — RSA-2048 + SHA-256 + AES-256 Key (k=256B, hLen=32B, |M|=32B)

lHash	PS	01	M
32B	158B	1B	32B

DB = 223 B (32+158+1+32)

00	mSeed	maskedDB
1B	32B	223B

EM = 256 B (1+32+223)

M →(OAEP)→ EM →(OS2IP)→ m →(RSA)→ c = m^e mod n

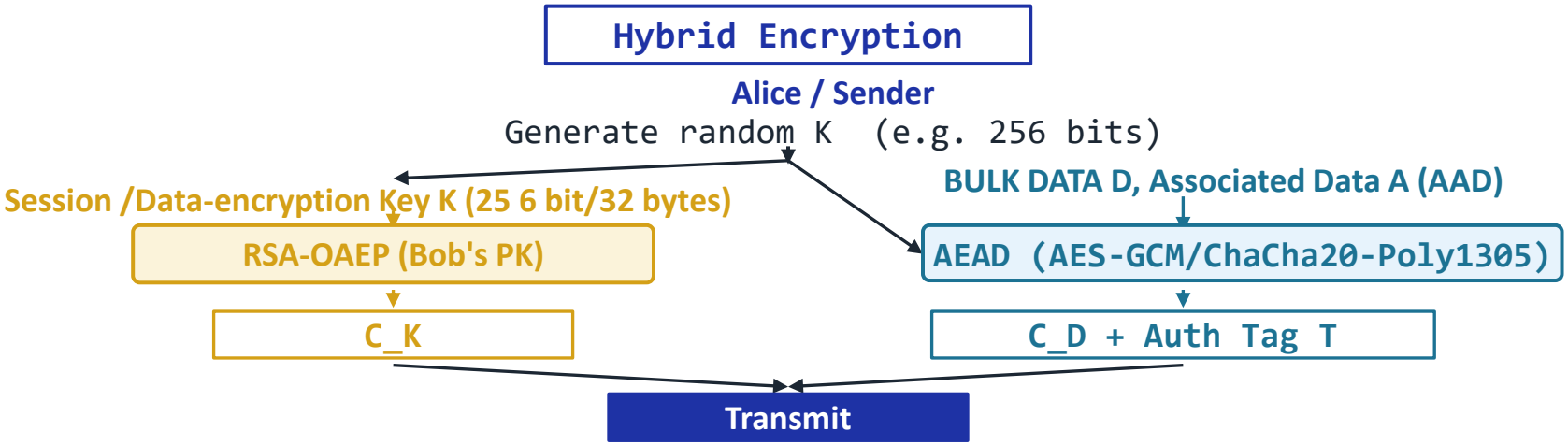
Octect String-to-Integer Primitive :OS2IP: octets → integer; leading 00 keeps m < n.

Receiver: reverse the masks (MGF1/XOR) → validate lHash and structure → M. Reversing XOR alone is not sufficient. NOTE: Construction intuition only; OAEP's formal security requires a proof (not covered in CS6530).

Key idea: OAEP combines structure, fresh randomness, MGF1 and reversible masking to create a randomized encoded message before the RSA operation.

RSA in Practice — Encrypt the Key, Not the Data

RSA-2048 + OAEP/SHA-256 can carry at most 190 bytes. How do we protect a large file or application message?




```
C_K = RSA-OAEP(PK_B, K)        (C_D, T) = AEAD.Enc(K, N, A, D)
Bob:  K = RSA-OAEP.Dec(SK_B, C_K)   D = AEAD.Dec(K, N, A, C_D, T)
```

WHY HYBRID?

RSA is expensive
Large-integer public-key ops are comparatively costly.

RSA-OAEP is limited
RSA-2048+SHA-256: M ≤ 190 B

AEAD suits bulk data
Confidentiality + Integrity/Auth

Think Like an Engineer

If K is already freshly random, why use OAEP?

Use RSA-OAEP — do not rely on properties of the plaintext.

***Protocol note**: Where used? RSA-OAEP is used for secure transport/wrapping of small secrets in RSA-based hybrid/envelope-encryption systems; modern TLS 1.3 instead uses ephemeral (EC)DH for key establishment.*

Key idea: Use RSA-OAEP to protect a small secret (Session Key); use symmetric AEAD to protect the bulk data.

RSA for Digital Signatures — A Different Security Goal

ENCRYPTION

Only the intended recipient should be able to recover the protected message.

`PK → encrypt · SK → decrypt`

DIGITAL SIGNATURE

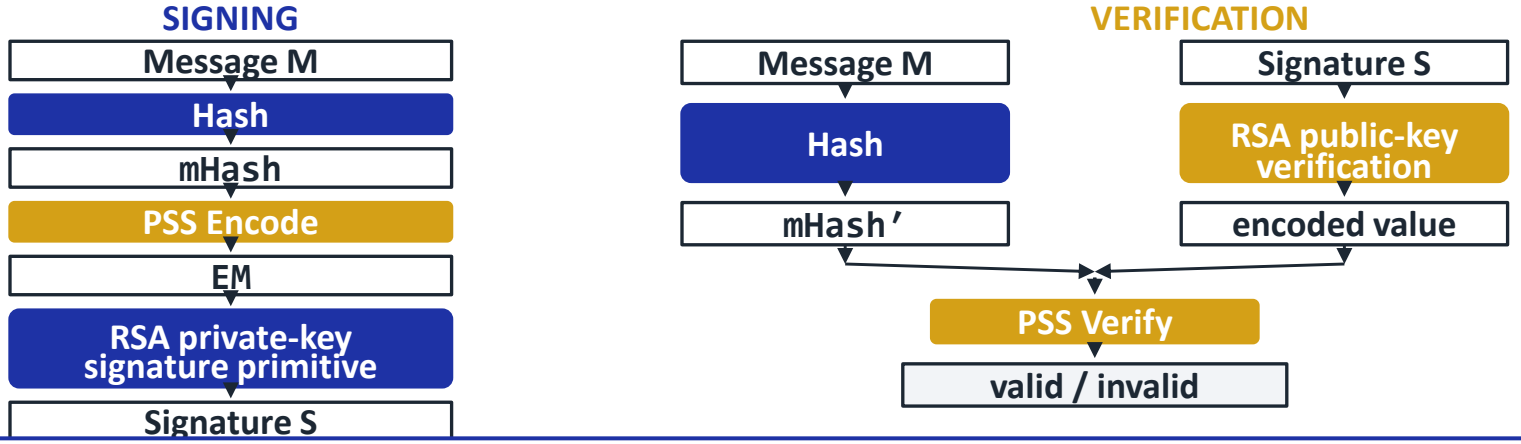
Verify that the message was signed by the holder of the private key and has not been altered.

`SK → sign · PK → verify`

COMMON SHORTCUT — BUT MISLEADING

`RSA signing is not simply "private-key encryption."`

The mathematical symmetry helps intuition, but a secure signature uses a defined signature encoding scheme.



`M →(Hash)→ mHash →(Probabilistic Signature Scheme (PSS))→ EM →(RSA private op)→ S`

Why Hash First?

Handles arbitrary-size messages · fixed-size digest · signature binds to the digest. RSA does not directly sign bulk data.

RSASP1 | RSAVP1

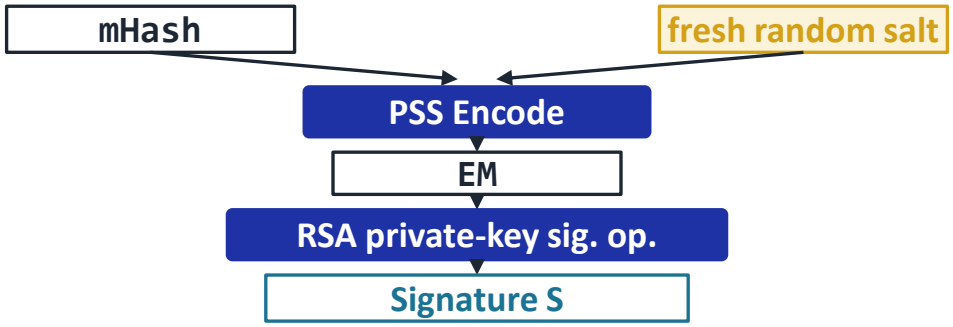
Signature/Verification Primitive 1 — shown for completeness only.

Key idea: RSA signatures apply a defined signature encoding to a message hash before the RSA private-key operation.

RSASSA-PSS — Randomize the Signature Encoding

OAEP introduced fresh randomness for RSA encryption. What does secure RSA signing do?

PSS combines the message hash with a fresh random salt.



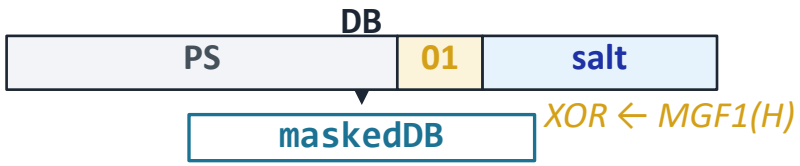
same M + fresh salt ⇒ different valid signatures can result

Inside PSS

$mHash = Hash(M)$ $salt \leftarrow\$ \{0,1\}^{8slen}$

$M' = 00...00 \parallel mHash \parallel salt$ $H = Hash(M')$

$DB = PS \parallel 01 \parallel salt$ $dbMask = MGF1(H, len)$ $maskedDB = DB \oplus dbMask$





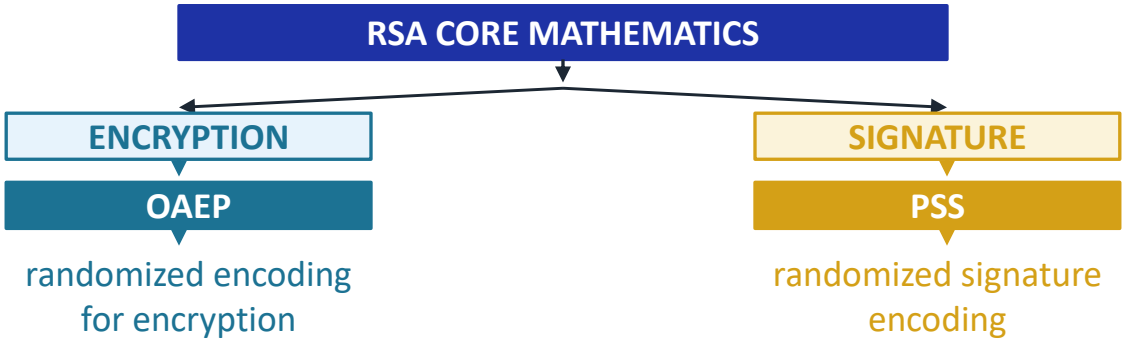
bc = 0xBC — trailer octet

The salt is random, not secret. *The verifier does not require a separately shared secret salt.*

Terminology: *seed/salt/nonce/IV name construction-specific roles, not secrecy guarantees. PSS also specifies bit-handling and verification detail omitted here for construction intuition.*

Key idea: RSASSA-PSS combines the message hash with fresh randomness and a defined encoding before applying the RSA signature primitive.

RSA-OAEP vs RSASSA-PSS — Same RSA, Different Constructions



OAEP and PSS are NOT interchangeable

	RSA-OAEP	RSASSA-PSS
Purpose	Encryption	Digital signature
Primary input	Message (often small secret)	Message hash
Fresh randomness	Seed	Salt
Hash used	Yes	Yes
MGF1 used	Yes	Yes
Produces	Encoded message (EM)	Encoded message (EM)
RSA side	Public-key encrypt	Private-key sign
Other side	Private-key decrypt	Public-key verify
Repeated input	Fresh seed → different C	Fresh salt → different valid S
Typical role	Protect a small secret/key	Authenticate/sign data

Application input → purpose-specific encoding → RSA primitive

INSIGHT: *Shared building blocks do not imply shared purpose — OAEP and PSS both use hashes, MGF1 and randomness, but their goals and rules differ.*

Key idea: OAEP and PSS are purpose-specific constructions around RSA — OAEP for encryption, PSS for signatures.

The Mathematics Is Secure —Secure RSA Can Still Fail in Implementation – (1/2)

WHAT IF TWO RSA KEYS SHARE A PRIME?

$n_1 = p \cdot q_1$ $\quad$ $n_2 = p \cdot q_2$

$\gcd(n_1, n_2) = p$

shared prime → factor both moduli

1 · RANDOMNESS FAILURE

$n_1 = pq_1,\ n_2 = pq_2 \Rightarrow \gcd(n_1, n_2) = p$

Poor RSA key-generation randomness can produce related/shared primes. Secure randomness is also required for the OAEP seed and PSS salt.

MITIGATION

CSPRNG + trusted key-generation implementation

CAN COMPUTATION TIME LEAK THE PRIVATE KEY?

RSA private op. uses secret d

C_1	8.1 ms
C_2	8.4 ms
C_3	7.9 ms

repeated observations → information?

2 · SIDE-CHANNEL LEAKAGE

correct result ≠ secret-independent behaviour

Timing, cache/memory, power or EM behaviour may correlate with secret-dependent private-key computation.

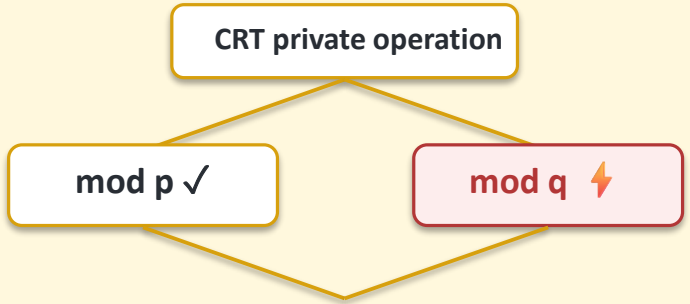
MITIGATION

Hardened/constant-time code where applicable · RSA blinding · reviewed libraries

Key idea: Attackers may not need to break RSA mathematics if the implementation gives them an easier path.

The Mathematics Is Secure —Secure RSA Can Still Fail in Implementation – (2/2)

WHAT IF FAST RSA SUFFERS A FAULT?



faulty signature

optimization can create a new attack surface

3 · CRT / FAULT ATTACKS

performance optimization → new security boundary

CRT speeds private RSA operations, but a fault in one branch can make key recovery possible.

MITIGATION

Fault detection · verify/check private-key results before release

WHAT IF THE SERVER REVEALS "VALID / INVALID"?



many chosen queries → learn from YES / NO

Bleichenbacher (1998): PKCS #1 v1.5 encryption

4 · DECRYPTION / PADDING ORACLES

Different externally observable decoding outcomes can turn a server into an oracle. Classic example: Bleichenbacher (1998), PKCS #1 v1.5 encryption — not OAEP.

MITIGATION

Hardened standardized implementations · appropriate uniform failure handling

Secure Algorithm + Secure Implementation + Secure Protocol Use = Secure Deployment

Key idea: RSA security depends on randomness, implementation behaviour and protocol handling — not only on the mathematics.

Secure Use of RSA — Six Rules to Remember

01 NEVER DEPLOY TEXTBOOK RSA

Primitive ≠ Scheme

02 USE PURPOSE-SPECIFIC SCHEMES

Encryption: RSA-OAEP
Signature: RSA-PSS

03 ENCRYPT KEYS, NOT BULK DATA

RSA-OAEP(K) + $AEAD_k$(Data)

04 CHOOSE APPROPRIATE PARAMETERS

RSA-1024: historical
RSA-2048 ≈112-bit
RSA-3072 ≈128-bit

05 PROTECT THE IMPLEMENTATION

Secure RNG · hardened libraries
side-channel + fault protection
safe failure handling

06 RSA IS NOT POST-QUANTUM SECURE

Shor + large-scale quantum computer breaks factoring-based RSA

PQC → later module

RSA MATHEMATICS — SECURE SCHEME — SECURE IMPLEMENTATION — SECURE DEPLOYMENT

Key idea: Use standardized RSA schemes, appropriate parameters and hardened implementations — and use RSA only for the job it is designed to perform.

References and Further Reading

STANDARDS / SPECIFICATIONS

RFC 8017 — PKCS #1 v2.2

PKCS #1: RSA Cryptography Specifications Version 2.2
Primary reference for RSA primitives, RSAES-OAEP, RSASSA-PSS and MGF1.

NIST SP 800-56B Rev. 2
Pair-Wise Key-Establishment Schemes Using Integer Factorization Cryptography

NIST SP 800-57 Part 1 Rev. 5
Recommendation for Key Management: Part 1 — General

FIPS 186-5
Digital Signature Standard (DSS)

PKCS = Public-Key Cryptography Standards
PKCS #1 is the RSA cryptography specification family.

FOUNDATIONAL / HISTORICAL

Goldwasser & Micali — Probabilistic Encryption
Journal of Computer and System Sciences, 1984

Bellare & Rogaway — Optimal Asymmetric Encryption — How to Encrypt with RSA
EUROCRYPT 1994

Bleichenbacher — Chosen Ciphertext Attacks Against Protocols Based on the RSA Encryption Standard PKCS #1
CRYPTO 1998

Rivest, Shamir & Adleman — A Method for Obtaining Digital Signatures and Public-Key Cryptosystems
CACM, 1978

COURSE TEXTS

Jean-Philippe Aumasson — Serious Cryptography, 2nd Ed.
Christof Paar & Jan Pelzl — Understanding Cryptography

RSA origin → RSA paper · Security notions → Goldwasser–Micali · OAEP → Bellare–Rogaway · Oracle attack → Bleichenbacher · Modern schemes → RFC 8017